

UNIVERSIDAD NACIONAL DE COLOMBIA

Inteligencia Artificial y Mini-Robots

Introducción a Machine Learning

Capítulo 6 · Solución de los ejercicios (sección 6.12)

Estudiantes:

David Orlando Tarazona Saldarriaga

Santiago Gómez Camargo

Juan David Rivera Buitrago

Docente:

José Martínez

Universidad Nacional de Colombia

Octubre de 2026

Contenido

6.12.0 Alcance, metodología y convenciones	3
Flujo de trabajo común	3
Convenciones y dos precisiones sobre el texto base	3
6.12.1 Ejercicio 1 · Multiplicación de matrices aprendida con ML	4
Planteamiento	4
Construcción del data set	4
Resultados en el conjunto de prueba (10.000 productos)	4
Uso del modelo con 10 ejemplos nuevos y comparación con el cálculo analítico	5
¿Qué es más costoso computacionalmente?	6
Profundización: generalización y cantidad de datos	7
6.12.2 Datos gubernamentales para los ejercicios 2, 3 y 4	8
Fuente y recolección	8
Limpieza, asignación de faltantes y transformación	8
Análisis exploratorio y sesgos detectados	9
Partición y línea base	9
6.12.3 Ejercicio 2 · Máquinas de Vectores de Soporte (SVM)	10
Documentación del algoritmo	10
Verificación de la implementación	11
Aplicación: riesgo de fallecimiento por COVID-19	11
Cambios introducidos para la aplicación	12
6.12.4 Ejercicio 3 · K vecinos más cercanos (K-NN)	13
Documentación del algoritmo	13
Verificación de la implementación	14
Cambios introducidos para la aplicación	14
6.12.5 Ejercicio 4 · Árboles de decisión	16
Documentación del algoritmo	16
Verificación de la implementación	16
Aplicación: riesgo de fallecimiento por COVID-19	17
Cambios introducidos para la aplicación	18
6.12.6 Comparación de los algoritmos y conclusiones	19
Comparación cualitativa	20
Conclusiones generales	20
Referencias	20
Anexo A · Código fuente documentado	22

6.12.0 Alcance, metodología y convenciones

El capítulo 6 cierra con cinco ejercicios y una instrucción explícita: resolver el ejercicio 1 y **tres** de los cuatro ejercicios siguientes, utilizando en alguno de ellos datos de una entidad gubernamental. Este documento resuelve el **ejercicio 1** (multiplicación de matrices 2×2 aprendida con ML) y los **ejercicios 2, 3 y 4** (SVM, K-NN y árboles de decisión). El ejercicio 5 (Bayes ingenuo) no se desarrolla, conforme a la instrucción; se escogieron los tres ejercicios que piden además *modificar* el algoritmo para una aplicación, que son los que exigen mayor profundidad.

Los tres algoritmos se **implementaron desde cero** en Python/NumPy, con documentación matemática en el propio código (Anexo A). Cada implementación se verificó contra scikit-learn en un data set de referencia antes de usarla, y luego se aplicó a un problema real con **datos abiertos del Instituto Nacional de Salud (INS)**, publicados en datos.gov.co en cumplimiento de la Ley 1712 de 2014 que el capítulo cita en la sección 6.8.2.

Flujo de trabajo común

Se siguieron los pasos de proyecto de la sección 6.10 y la figura 6.14 del capítulo:

- **Objetivo y características:** se define la tarea T, la experiencia E y la medida P (definición de Mitchell, sección 6.2).
- **Criterio de evaluación:** matriz de confusión (figura 6.2) y medidas derivadas; validación cruzada estratificada de 5 pliegues.
- **Solución actual:** una regla escrita a mano («programación tradicional», figura 6.1) sirve de línea base.
- **Datos:** recolección, limpieza, asignación de faltantes y transformación (secciones 6.7.2 a 6.7.4), con revisión explícita de sesgos (6.7.6).
- **Modelo y análisis de errores:** selección de hiperparámetros por validación cruzada y evaluación final *una sola vez* en el conjunto de prueba.

Convenciones y dos precisiones sobre el texto base

Nota 1 · «Precisión» versus exactitud

La fórmula que el capítulo llama *precisión*, $(VP + VN)/(VP + VN + FP + FN)$, en la literatura se denomina **exactitud** (*accuracy*). En este documento se reserva la palabra **precisión** para $VP/(VP + FP)$, que es el uso estándar en scikit-learn y en la literatura médica (donde también se llama valor predictivo positivo). Las medidas usadas son:

$$\text{Exactitud} = \frac{VP + VN}{n} \quad \text{Sensibilidad} = \frac{VP}{VP + FN} \quad \text{Especificidad} = \frac{VN}{VN + FP} \quad \text{Precisión} = \frac{VP}{VP + FP}$$

Nota 2 · Síntoma del sobreentrenamiento

La sección 6.8.1 indica que el modelo está sobreentrenado «cuando la precisión del conjunto de validación es mayor que la obtenida con el conjunto de entrenamiento». El síntoma habitual es el **opuesto**: el modelo acierta mucho más en entrenamiento que en validación porque memorizó el ruido. El ejercicio 4 lo muestra con claridad: el árbol sin podar acierta el 99,3 % en entrenamiento y solo el 76,8 % en validación cruzada.

Reproducibilidad. Todas las cifras de este documento provienen de ejecutar el código del Anexo A con semillas fijas (42 para particiones y modelos; 2026 para los 10 ejemplos del ejercicio 1). Entorno: Python 3.11, NumPy 2.4, scikit-learn 1.8, 2 núcleos de CPU. Los números decimales se escriben con coma y los miles con punto.

6.12.1 Ejercicio 1 · Multiplicación de matrices aprendida con ML

Enunciado

Considere la figura 6.1. Conforme un data set con resultados de 50.000 multiplicaciones de matrices 2×2 , generadas aleatoriamente con números enteros entre -20 y 20 . Una vez entrenado el modelo, úselo con 10 ejemplos y compare los resultados con los que se producen al realizar la operación analíticamente. Haga un cálculo de qué es más costoso computacionalmente.

Planteamiento

La figura 6.1 contrasta los dos paradigmas: en programación tradicional se entregan *datos + reglas* y se obtienen respuestas; en ML se entregan *datos + respuestas* y se obtienen las reglas (el modelo). Aquí la regla es conocida:

$$C = AB, \quad c_{ij} = \sum_{k=1}^2 a_{ik} b_{kj} \Rightarrow c_{11} = a_{11}b_{11} + a_{12}b_{21}, \quad c_{12} = a_{11}b_{12} + a_{12}b_{22}, \quad \dots$$

Como problema de ML supervisado es una **regresión multisalida**: la entrada es el vector de 8 enteros $x = (a_{11}, a_{12}, a_{21}, a_{22}, b_{11}, b_{12}, b_{21}, b_{22})$ y la salida los 4 elementos de C . La tarea T es predecir C ; la experiencia E son los 50.000 pares; la medida P es el error absoluto medio (MAE) y el porcentaje de elementos acertados exactamente tras redondear al entero más cercano.

La observación clave para elegir modelos es que cada c_{ij} es **bilineal**: una suma de productos de dos entradas. Eso anticipa tres comportamientos distintos:

- **Regresión lineal** ($\hat{y} = Wx + b$): no puede representar productos; con entradas simétricas alrededor de cero, la mejor aproximación lineal es prácticamente la constante 0. Sirve de control negativo.
- **Regresión polinomial de grado 2**: agrega los 36 productos $x_p x_q$ como características (44 en total). La regla verdadera está *dentro* del espacio de hipótesis, así que mínimos cuadrados puede encontrarla exactamente.
- **Red neuronal MLP 8-128-128-4** (ReLU, Adam, parada temprana): aproximador universal que no conoce la estructura del problema; debe aprenderla de los datos, como en los ejemplos de redes del capítulo.

Construcción del data set

Se generaron 50.000 pares (A, B) con enteros uniformes en $[-20, 20]$ y se calculó $C = AB$ para obtener las respuestas correctas. Los elementos de C quedan en $[-800, 800]$. El data set se partió en 40.000 ejemplos de entrenamiento y 10.000 de prueba (80/20); la MLP reservó a su vez el 10 % del entrenamiento para validación y parada temprana. Para la red, las entradas se estandarizaron (media 0, desviación 1) y las salidas se dividieron entre 800.

Resultados en el conjunto de prueba (10.000 productos)

Tabla 6.12.1. Desempeño de los tres modelos en los 10.000 productos de prueba.

Modelo	MAE	RMSE	R ²	Elementos exactos	Matrices exactas	Entrenamiento
Regresión lineal	154,82	197,57	-0,0002	0,24 %	0,00 %	0,03 s
Regresión polinomial (grado 2)	$2,22 \cdot 10^{-13}$	$2,98 \cdot 10^{-13}$	1,0000	100,00 %	100,00 %	0,09 s
Red neuronal MLP 8-128-128-4	2,80	3,60	0,9997	11,65 %	0,02 %	132,79 s (396 épocas)

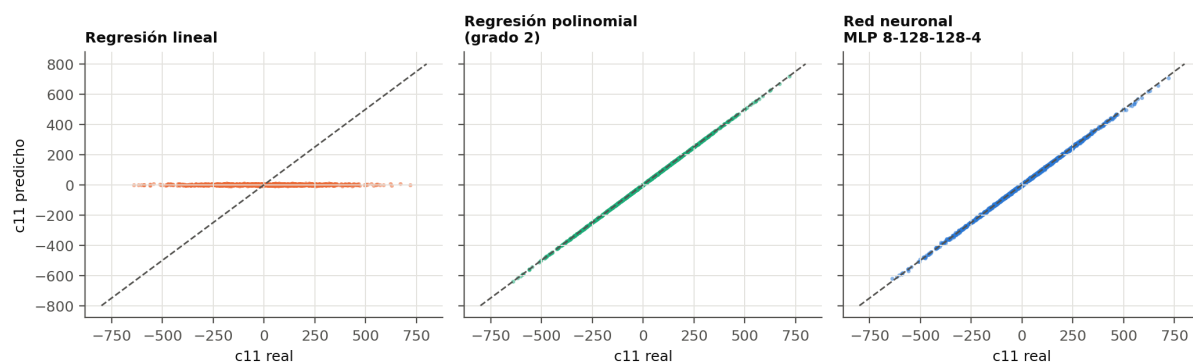


Figura 6.12.1. Elemento c_{11} predicho contra el real en 1.500 productos de prueba. La regresión lineal predice ≈ 0 para todo; la polinomial cae exactamente sobre la diagonal; la MLP la sigue con pequeñas desviaciones.

El modelo polinomial «descubre» la regla

Al inspeccionar los 44 coeficientes aprendidos para cada salida, solo dos son distintos de cero y valen exactamente 1; los demás son del orden de 10^{-15} (error de redondeo de punto flotante). Es decir, el modelo recuperó la definición de producto matricial a partir de ejemplos, que es literalmente la flecha «Rules (model)» de la figura 6.1:

Tabla 6.12.2. Regla aprendida por la regresión polinomial de grado 2.

Salida	Términos con coeficiente ≈ 1	Mayor coeficiente restante (en valor absoluto)
c11	$1,000000 \cdot a_{12} \cdot b_{21} + 1,000000 \cdot a_{11} \cdot b_{11}$	$3,3 \cdot 10^{-15}$
c12	$1,000000 \cdot a_{11} \cdot b_{12} + 1,000000 \cdot a_{12} \cdot b_{22}$	$2,3 \cdot 10^{-15}$
c21	$1,000000 \cdot a_{22} \cdot b_{21} + 1,000000 \cdot a_{21} \cdot b_{11}$	$1,6 \cdot 10^{-15}$
c22	$1,000000 \cdot a_{21} \cdot b_{12} + 1,000000 \cdot a_{22} \cdot b_{22}$	$2,3 \cdot 10^{-15}$

Uso del modelo con 10 ejemplos nuevos y comparación con el cálculo analítico

Se generaron 10 pares nuevos (semilla 2026), que ningún modelo vio al entrenar. La tabla muestra cada matriz con sus dos filas; se incluyen el resultado analítico y las predicciones. Ambas predicciones se muestran con dos decimales.

Tabla 6.12.3. Diez ejemplos nuevos: resultado analítico frente a las predicciones.

#	A	B	C = AB (analítico)	Regresión polinomial	MLP	Error máx. MLP
1	[14 -13] [-19 6]	[15 1] [-17 -3]	[431 53] [-387 -37]	[431,00 53,00] [-387,00 -37,00]	[432,44 53,20] [-387,26 -34,20]	2,80
2	[-6 -1] [-17 -5]	[-9 7] [14 -20]	[40 -22] [83 -19]	[40,00 -22,00] [83,00 -19,00]	[45,26 -20,25] [80,63 -22,16]	5,26
3	[6 -6] [14 12]	[-1 -2] [20 -6]	[-126 24] [226 -100]	[-126,00 24,00] [226,00 -100,00]	[-124,74 24,08] [227,20 -100,14]	1,26
4	[8 17] [9 -13]	[-14 -12] [-1 4]	[-129 -28] [-113 -160]	[-129,00 -28,00] [-113,00 -160,00]	[-134,16 -27,80] [-113,18 -161,30]	5,16
5	[15 6] [-16 -8]	[-7 -3] [7 -8]	[-63 -93] [56 112]	[-63,00 -93,00] [56,00 112,00]	[-63,77 -88,47] [57,11 113,10]	4,53
6	[-14 19] [9 17]	[19 -12] [18 15]	[76 453] [477 147]	[76,00 453,00] [477,00 147,00]	[71,47 453,04] [480,20 148,84]	4,53
7	[-9 6] [4 10]	[19 12] [1 4]	[-165 -84] [86 88]	[-165,00 -84,00] [86,00 88,00]	[-160,09 -84,69] [89,26 90,40]	4,91
8	[-16 1] [6 13]	[-12 -6] [-2 18]	[190 114] [-98 198]	[190,00 114,00] [-98,00 198,00]	[194,82 114,69] [-94,20 194,84]	4,82
9	[6 -2] [-2 -7]	[12 3] [4 -3]	[64 24] [-52 15]	[64,00 24,00] [-52,00 15,00]	[63,76 25,56] [-51,16 17,28]	2,28

#	A	B	C = AB (analítico)	Regresión polinomial	MLP	Error máx. MLP
10	$\begin{bmatrix} -14 & -9 \\ -15 & -11 \end{bmatrix}$	$\begin{bmatrix} -3 & 16 \\ -20 & -7 \end{bmatrix}$	$\begin{bmatrix} 222 & -161 \\ 265 & -163 \end{bmatrix}$	$\begin{bmatrix} 222,00 & -161,00 \\ 265,00 & -163,00 \end{bmatrix}$	$\begin{bmatrix} 225,10 & -163,88 \\ 264,26 & -160,39 \end{bmatrix}$	3,10

En estos 10 ejemplos la regresión polinomial acertó los 40 elementos ($MAE = 2,19 \cdot 10^{-13}$). La MLP obtuvo $MAE = 2,05$ y $R^2 = 0,9997$: una aproximación muy buena en términos estadísticos, pero solo el 20 % de los elementos coincide al redondear y **ninguna** de las 10 matrices sale completamente correcta. La regresión lineal ($MAE = 136,7$) no sirve. Para una operación que exige exactitud entera, «casi correcto» equivale a incorrecto.

¿Qué es más costoso computacionalmente?

El costo se analizó de tres formas: conteo teórico de operaciones de punto flotante (FLOPs) por producto, tiempo medido de inferencia y costo de entrenamiento.

Tabla 6.12.4. Costo teórico de inferencia por cada producto de matrices.

Método	Operaciones por producto	Detalle del conteo	Relación vs. analítico
Analítico (definición)	12	8 multiplicaciones + 4 sumas	1×
Regresión polinomial	392	36 productos de características + 4 salidas × 44 multiplicaciones-sumas	33×
MLP 8-128-128-4	36.372	$8 \cdot 128 + 128 \cdot 128 + 128 \cdot 4 = 17.920$ multiplicaciones-sumas, sesgos, ReLU y escalado	3.031×

Tabla 6.12.5. Tiempo medido de inferencia (mediana de varias repeticiones).

Método	10 productos	100.000 productos	Relación vs. NumPy (100.000)
Analítico (Python puro)	2,6 μ s	179,3 ms	49,4×
Analítico (NumPy)	2,5 μ s	3,6 ms	1,0×
Regresión polinomial	215,0 μ s	39,0 ms	10,7×
MLP	252,1 μ s	609,9 ms	168,2×

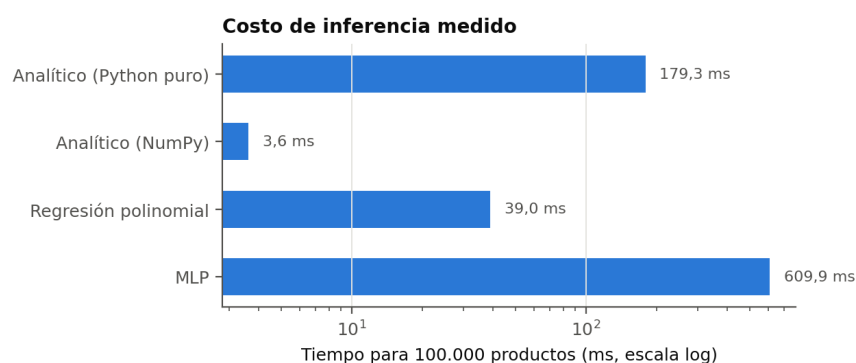


Figura 6.12.2. Tiempo para calcular 100.000 productos (escala logarítmica).

Entrenamiento. El cálculo analítico no requiere entrenamiento. La regresión polinomial resuelve un sistema de mínimos cuadrados de 44 incógnitas (del orden de $1,5 \cdot 10^8$ FLOPs, 0,09 s). La MLP necesitó 396 épocas sobre 36.000 ejemplos; con el factor habitual de 3× (propagación hacia adelante y hacia atrás) son unos $1,6 \cdot 10^{12}$ FLOPs y 133 s de reloj. Con ese mismo presupuesto se podrían calcular analíticamente del orden de $1 \cdot 10^{11}$ productos de matrices.

Respuesta: el cálculo analítico es, con amplio margen, el menos costoso

Por producto, la MLP requiere unas 3.031 veces más operaciones que la definición y la polinomial unas 33 veces más. En tiempo medido (100.000 productos) la MLP fue 168 veces más lenta que NumPy y la polinomial 11 veces. A esto se suma el costo de generar 50.000 respuestas (¡calculadas analíticamente!) y de entrenar. Con 10 productos los tiempos de los modelos están dominados por la sobrecarga fija de la librería ($\approx 0,2$ ms), no por la aritmética.

La conclusión coincide con la sección 6.4 del capítulo: ML sobresale cuando no existe un algoritmo conocido o este es demasiado complejo. Cuando la regla es conocida y barata, programarla directamente es más exacto y más eficiente.

Profundización: generalización y cantidad de datos

Extrapolación. Se evaluaron ambos modelos con 5.000 productos de enteros en $[-100, 100]$, fuera del rango de entrenamiento. La polinomial siguió siendo exacta ($\text{MAE} = 5,48 \cdot 10^{-12}$) porque aprendió la regla y no los datos. La MLP colapsó: $\text{MAE} = 2.340$ y $R^2 = 0,58$. Con activaciones ReLU la red es lineal por tramos y fuera del rango visto solo puede prolongar esos tramos, que no reproducen una función cuadrática. Es un ejemplo directo del aviso de la sección 6.4: el data set puede no contener todo el conocimiento necesario.

Curva de aprendizaje. Al entrenar con subconjuntos de 500 a 40.000 ejemplos, el error de la MLP bajó de 36,2 a 2,8, aproximadamente como $n^{-0,56}$ (ajuste log-log). La polinomial ya es exacta con 500 ejemplos: basta con más ecuaciones que incógnitas (44). Es la «irrazonable efectividad de los datos» de la sección 6.7 vista desde el otro lado: más datos ayudan al modelo genérico, pero el conocimiento previo correcto (la forma cuadrática) vale más que decenas de miles de ejemplos.

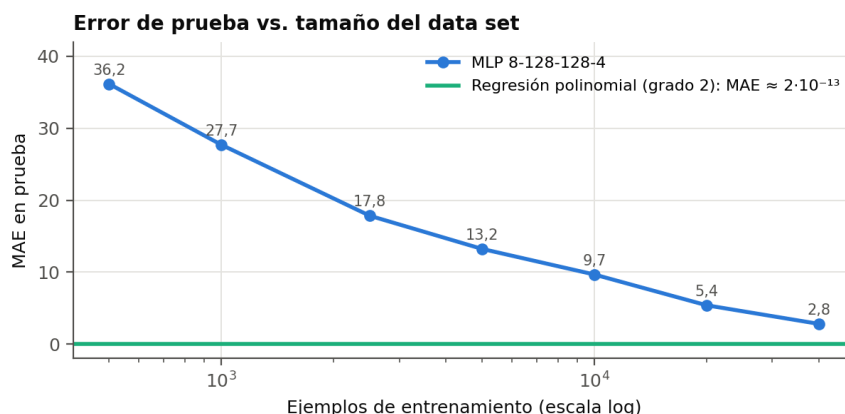


Figura 6.12.3. MAE de prueba de la MLP según el número de ejemplos de entrenamiento. La regresión polinomial es exacta en todos los tamaños.

Conclusiones del ejercicio 1

- El mismo problema puede ser imposible (lineal), trivial (polinomial) o aproximable con esfuerzo (MLP): la elección del espacio de hipótesis domina el resultado.
- Una métrica agregada excelente ($R^2 = 0,9997$) puede ocultar que ninguna respuesta es exactamente correcta. La métrica debe responder a la necesidad del problema.
- Aprender una operación con regla conocida es mucho más caro que ejecutarla; el valor didáctico del ejercicio está en ver a ML descubrir la regla, no en reemplazarla.

6.12.2 Datos gubernamentales para los ejercicios 2, 3 y 4

Para cumplir el requisito de usar datos de una entidad gubernamental, los tres algoritmos se aplicaron al mismo problema, lo que además permite compararlos en igualdad de condiciones (sección 6.12.6).

Fuente y recolección

Se usó el conjunto «**Casos positivos de COVID-19 en Colombia**» del Instituto Nacional de Salud, publicado en el portal de datos abiertos del Estado (datos.gov.co, recurso gt2j-8ykr). Cada registro es un caso confirmado, anonimizado, con edad, sexo, tipo de contagio, departamento, fechas de inicio de síntomas y de diagnóstico, y el desenlace. Es una fuente *secundaria* en el sentido de la sección 6.7.2.1: el INS ya recopiló los datos.

Tarea T: clasificar un caso como **fallecido** (clase positiva, $y = 1$) o **recuperado** ($y = 0$) a partir de información disponible al momento del diagnóstico. Es un problema de triaje: identificar temprano a quién vigilar más.

Diseño de la muestra. Se consultó la API del portal filtrando por desenlace y departamento, y se tomaron 100 fallecidos y 100 recuperados de cada uno de cuatro departamentos (Bogotá D.C., Antioquia, Valle del Cauca y Atlántico): **800 casos**, balanceados 50/50. El balanceo es deliberado (submuestreo de la clase mayoritaria) porque en la población los fallecidos son una fracción pequeña de los casos (del orden del 2 %); con datos tan desbalanceados un clasificador que diga siempre «recuperado» tendría 98 % de exactitud sin servir para nada. La sección 6.12.3 muestra cómo corregir las probabilidades para volver a la prevalencia real.

Limpieza, asignación de faltantes y transformación

Tabla 6.12.6. Variables del modelo y su preparación.

Variable	Origen	Tratamiento (sección del capítulo)
edad	edad + unidad_medida	Conversión a años (unidad 2 = meses, 3 = días). Numérica.
sexo_M	sexo	Binaria: 1 masculino, 0 femenino (se normalizan minúsculas).
fuentes_relacionado	fuentes_tipo_contagio	Binaria: 1 si el contagio fue «Relacionado» con otro caso, 0 si «Comunitaria».
asintomatico	fecha_inicio_sintomas	Faltante informativo: 11 casos sin fecha de síntomas → 1 (6.7.3.2).
demora_dx	fechas	Días entre síntomas y diagnóstico; faltantes asignados con la mediana del entrenamiento (6.7.3.2.2); recorte a [0, 60].
dep_*	departamento	Codificación one-hot en 4 columnas (6.7.4). Para K-NN con HEOM: una sola columna categórica.

Se descartaron deliberadamente las columnas *estado* y *ubicación* (valores como «Fallecido» o «Casa») porque contienen la respuesta: usarlas sería una **fuga de información** y daría una exactitud engañosa cercana al 100 %.

Análisis exploratorio y sesgos detectados

Tabla 6.12.7. Estadística descriptiva por clase (análisis exploratorio, sección 6.7.3.1).

Indicador	Recuperados (n = 400)	Fallecidos (n = 400)
Edad media / mediana (años)	39,8 / 35	70,5 / 71
Hombres	53,8 %	62,3 %
Contagio relacionado	26,5 %	12,8 %
Asintomáticos	2,8 %	0,0 %
Demora síntomas → diagnóstico (días, media)	10,2	10,2
Demora exactamente igual a 15 días	20,5 %	11,0 %
Diagnóstico desde febrero de 2021	10,5 %	0,0 %

La edad separa claramente las clases (≈ 30 años de diferencia en la media): será la variable dominante. El análisis exploratorio reveló además tres problemas de calidad que se documentan porque el capítulo (6.7.6 y 6.11) insiste en ellos:

- **Sesgo temporal de muestreo.** La API devuelve los registros en su orden interno: los fallecidos tomados son de 2020 y comienzos de 2021, mientras que parte de los recuperados es de finales de 2021 y 2022. Un modelo con la fecha como variable aprendería «época» y no «riesgo». Por eso **las fechas no se usan** como característica.
- **Valores imputados en la fuente.** En Antioquia muchos recuperados tienen fecha de síntomas exactamente 15 días antes del diagnóstico (20,5 % de los recuperados frente a 11,0 % de los fallecidos). Es un rastro de imputación administrativa, no un fenómeno clínico: un sesgo de medición que los modelos pueden explotar (se verá en K-NN y árboles).
- **Sesgo de exclusión.** Los casos con desenlace «N/A» o activos no están en la muestra; el modelo solo aprende a distinguir entre los dos desenlaces cerrados.

Partición y línea base

Partición estratificada 75/25: **600 casos de entrenamiento** y **200 de prueba**, ambos balanceados. Toda decisión (hiperparámetros, variantes, poda) se tomó con **validación cruzada estratificada de 5 pliegues dentro del entrenamiento**; el conjunto de prueba se usó una sola vez al final (esquema de la sección 6.8.1).

Línea base «tradicional». Antes de cualquier ML se buscó la mejor regla de una sola condición escrita a mano: «si edad $\geq u$ entonces fallecido». El mejor umbral en entrenamiento fue **$u = 54$ años** (exactitud de entrenamiento 83,8 %); en prueba obtuvo exactitud 81,0 %, sensibilidad 90,0 % y especificidad 72,0 %. Todo modelo de ML debe compararse contra esto.

6.12.3 Ejercicio 2 · Máquinas de Vectores de Soporte (SVM)

Enunciado

Estudie el algoritmo SVM con todo detalle, mejore su documentación y con base en él haga cambios para una aplicación.

Documentación del algoritmo

Idea: el hiperplano de margen máximo

Para dos clases separables existen infinitos hiperplanos $w \cdot x + b = 0$ que las separan. La SVM (Cortes y Vapnik, 1995) escoge el que deja la **mayor distancia** (margen) al punto más cercano de cada clase. Si se escalan w y b para que los puntos más cercanos cumplan $|w \cdot x + b| = 1$, el ancho del margen es $2/\|w\|$; maximizarlo equivale a minimizar $\|w\|^2$. Los puntos que quedan sobre el borde del margen (o dentro de él) son los **vectores de soporte**: son los únicos que determinan la frontera; los demás podrían eliminarse sin cambiar nada.

Margen suave (datos reales no separables)

Se introducen holguras $\xi_i \geq 0$ que permiten violar el margen pagando un costo C . Aquí se generaliza a un costo por ejemplo C_i , base del primer cambio para la aplicación:

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + \sum_{i=1}^n C_i \xi_i \quad \text{s. a.} \quad y_i (w \cdot \phi(x_i) + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad y_i \in \{-1, +1\}$$

Equivalentemente, se minimiza $\frac{1}{2} \|w\|^2 + \sum C_i \max(0, 1 - y_i f(x_i))$: la **pérdida bisagra** (*hinge*) no castiga a los puntos bien clasificados fuera del margen y castiga linealmente a los demás. **C grande** = casi no se toleran errores de entrenamiento (margen estrecho, riesgo de sobreajuste); **C pequeño** = margen ancho y modelo más simple.

Problema dual y truco del núcleo

Con multiplicadores de Lagrange α_i el problema se reescribe en forma dual, que depende de los datos solo a través de productos internos $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$:

$$\min_{\alpha} \frac{1}{2} \alpha^T Q \alpha - e^T \alpha \quad \text{s. a.} \quad 0 \leq \alpha_i \leq C_i, \quad y^T \alpha = 0, \quad Q_{ij} = y_i y_j K(x_i, x_j)$$

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b, \quad \hat{y} = \text{signo}(f(x)), \quad K_{RBF}(x, z) = \exp(-\gamma \|x - z\|^2)$$

El **truco del núcleo** permite trabajar en un espacio de características ϕ de dimensión enorme (infinita para el núcleo RBF) sin calcular ϕ explícitamente. Con el núcleo lineal $K = x \cdot z$ se recupera la SVM lineal. En el RBF, γ controla el alcance de cada ejemplo: γ grande $\Rightarrow$ fronteras muy irregulares (sobreajuste); γ pequeño $\Rightarrow$ fronteras suaves, casi lineales. Las condiciones de Karush-Kuhn-Tucker (KKT) caracterizan la solución: $\alpha_i = 0$ para puntos bien clasificados fuera del margen, $0 < \alpha_i < C_i$ para puntos exactamente sobre el margen y $\alpha_i = C_i$ para los que lo violan.

Solucionador: SMO con selección del par de máxima violación

La implementación resuelve el dual con **Sequential Minimal Optimization** (Platt, 1998), en la variante de LIBSVM (Keerthi et al., 2001; Fan, Chen y Lin, 2005). La restricción $y \cdot \alpha = 0$ impide mover un solo α , así que en cada iteración:

- Se calcula el gradiente $G = Q\alpha - e$ (se mantiene actualizado en $O(n)$ por iteración).
- Se elige i con máximo $-y_i G_i$ entre los que pueden «subir» y j con mínimo $-y_j G_j$ entre los que pueden «bajar»: el par que más viola las condiciones KKT.

- Si la violación es menor que la tolerancia (10^{-3}), la solución es óptima y se termina.
- Se optimizan α_i y α_j **analíticamente** (es una parábola en una variable) y se recortan a la caja $[0, C_i]$.
- El sesgo b se obtiene promediando sobre los vectores de soporte libres ($0 < \alpha < C$).

Costo: memoria $O(n^2)$ para la matriz de núcleo y tiempo típico entre $O(n^2)$ y $O(n^3)$; adecuado hasta decenas de miles de ejemplos. Las características deben estar en escalas comparables (se estandarizan) porque el núcleo usa distancias.

Verificación de la implementación

Tabla 6.12.8. SVM propia frente a `sklearn.svm.SVC` en el data set Breast Cancer Wisconsin (569 casos, 30 variables).

Núcleo	C	Vectores de soporte (propio / sklearn)	b propio	b sklearn	Máx. $ f_{\text{propio}} - f_{\text{sk}} $	Acuerdo
lineal	0,1	60 / 60	0,2164	0,2166	0,0024	100 %
lineal	1	40 / 40	0,0444	0,0444	0,0277	100 %
lineal	10	37 / 37	-0,3084	-0,3085	0,0318	100 %
rbf ($\gamma = 0,05$)	0,1	256 / 256	-0,1921	-0,1921	0,0011	100 %
rbf ($\gamma = 0,05$)	1	146 / 146	-0,2287	-0,2287	0,0008	100 %
rbf ($\gamma = 0,05$)	10	122 / 122	-0,1839	-0,1839	0,0006	100 %

Los vectores de soporte coinciden en número, el sesgo coincide hasta la cuarta cifra decimal y todas las predicciones son iguales. Las pequeñas diferencias en $f(x)$ se deben a la tolerancia de parada.

Aplicación: riesgo de fallecimiento por COVID-19

Se estandarizaron las 9 variables y se buscaron hiperparámetros por validación cruzada. Con núcleo lineal la exactitud CV varió entre 83,2 % y 84,8 % para C entre 0,01 y 100. Con núcleo RBF se exploraron 20 combinaciones (figura 6.12.4); la mejor fue $C = 0,1$, $\gamma = 0,03$, con exactitud CV de 84,8 % $\pm 1,7$ %.

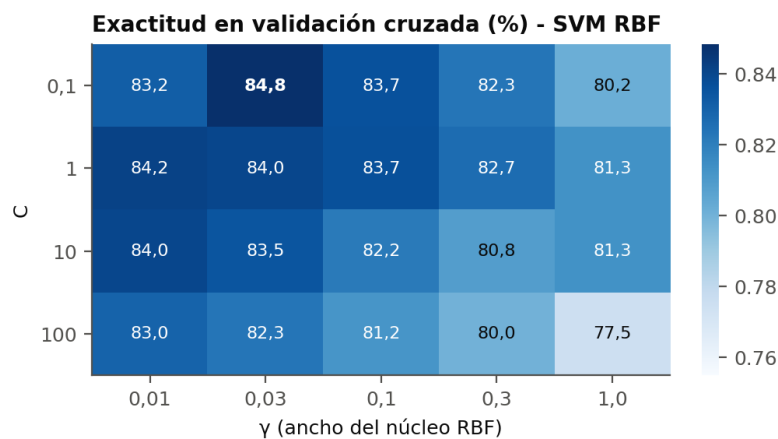


Figura 6.12.4. Exactitud en validación cruzada de la SVM-RBF para cada combinación de C y γ . Valores altos de γ (fronteras irregulares) empeoran el resultado.

El modelo final se entrenó con los 600 casos de entrenamiento: SMO convergió en 254 iteraciones (16 ms) y retuvo 468 vectores de soporte. Coincide al 100 % con `sklearn`. Resultado en los 200 casos de prueba:

	Predice: Fallecido	Predice: Recuperado
Real: Fallecido	VP = 91	FN = 9
Real: Recuperado	FP = 26	VN = 74

Exactitud **82,5 %** · sensibilidad **91,0 %** · especificidad **74,0 %** · precisión 77,8 % · F1 0,839

La SVM lineal ($C = 0,01$) obtiene resultados parecidos y es interpretable: en variables estandarizadas, los pesos más grandes en valor absoluto son *edad* (0,99), *fuentes_relacionado* (-0,19), *sexo_M* (0,12), *asintomatico* (-0,12). La edad pesa unas cinco veces más que cualquier otra variable; ser hombre aumenta el riesgo y el contagio «relacionado» lo disminuye (posiblemente porque esos casos se detectan antes gracias al rastreo de contactos).

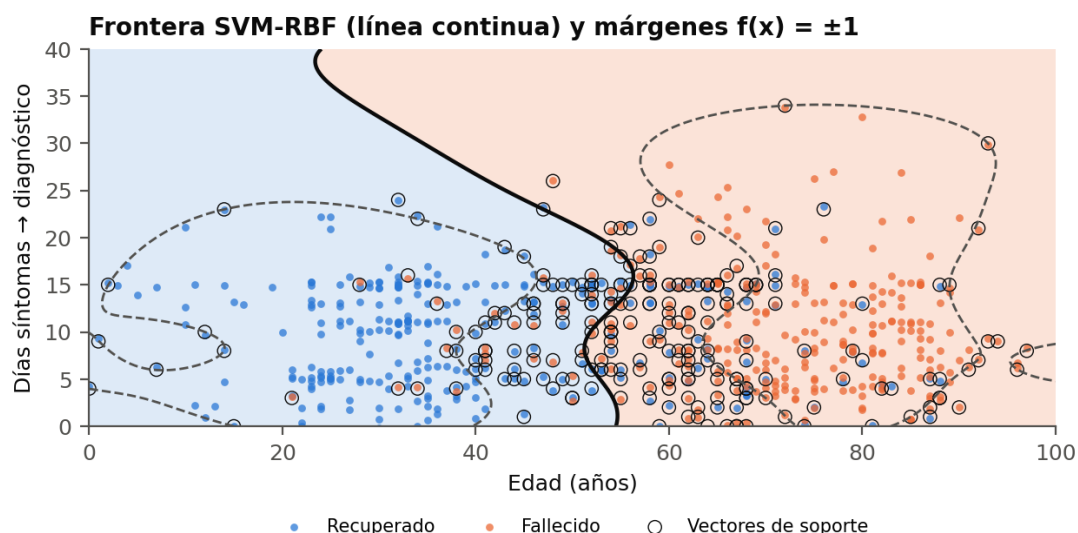


Figura 6.12.5. SVM-RBF entrenada solo con edad y demora del diagnóstico, para visualizar la frontera (línea continua), los márgenes $f(x) = \pm 1$ (discontinuas) y los vectores de soporte (círculos). Se aprecia la banda horizontal en 15 días que produce la imputación de la fuente.

Cambios introducidos para la aplicación

Cambio 1 · Costo asimétrico por clase

En triaje, un falso negativo (decir «bajo riesgo» a quien fallecerá) es más grave que un falso positivo (vigilar de más a quien se recuperará). La SVM estándar trata ambos errores igual. Se modificó el solucionador para aceptar $C_i = C \cdot \text{peso}[y_i]$: el límite superior de la caja de cada α depende de su clase, y la actualización analítica de SMO se ajustó para respetar límites distintos en α_i y α_j .

Tabla 6.12.9. Efecto del costo asimétrico en el conjunto de prueba.

Peso de la clase «fallecido»	Exactitud	Sensibilidad	Especificidad	FN	FP
1,0	82,5 %	91,0 %	74,0 %	9	26
1,5	80,5 %	91,0 %	70,0 %	9	30
2,0	79,0 %	95,0 %	63,0 %	5	37
3,0	75,5 %	99,0 %	52,0 %	1	48
4,0	69,5 %	99,0 %	40,0 %	1	60

Con peso 3 solo queda **un** fallecido sin detectar entre 100, a cambio de duplicar las falsas alarmas. La exactitud baja, lo que muestra por qué no debe ser la única medida: el peso correcto lo decide el costo real de cada error, no el algoritmo.

Cambio 2 · Salida probabilística (escalado de Platt)

La SVM entrega un puntaje $f(\mathbf{x})$ sin interpretación probabilística. Se añadió el método de Platt (1999): se ajusta una sigmoide sobre los puntajes por máxima verosimilitud con Newton-Raphson. Para no sobreajustar, los puntajes usados son *fuera de pliegue* (validación cruzada). Resultado: $A = -3,101$, $B = 0,232$.

$$P(y = 1 | \mathbf{x}) = \frac{1}{1 + \exp(A f(\mathbf{x}) + B)}$$

Cambio 3 · Corrección por prevalencia real

El modelo se entrenó con 50 % de fallecidos, pero la letalidad real es mucho menor. Por el teorema de Bayes, cambiar la proporción de clases multiplica las razones de probabilidad (*odds*) por un factor constante; se implementó esta corrección con $\pi_0 = 0,5$ (entrenamiento) y $\pi = 0,02$ (valor ilustrativo del orden de la letalidad observada):

$$\frac{p'}{1-p'} = \frac{p}{1-p} \cdot \frac{\pi/(1-\pi)}{\pi_0/(1-\pi_0)}$$

Tabla 6.12.10. Probabilidades estimadas para perfiles hipotéticos (contagio comunitario, sintomático, 8 días de demora).

Perfil (sin otros factores)	$f(\mathbf{x})$	P(fallece) con muestra 50/50	P(fallece) corregida a $\pi = 2$ %
25 años, mujer, Bogotá	-1,07	2,8 %	0,06 %
45 años, hombre, Antioquia	-0,22	28,4 %	0,80 %
60 años, mujer, Valle	0,08	50,7 %	2,06 %
70 años, hombre, Bogotá	0,82	91,1 %	17,20 %
85 años, mujer, Atlántico	1,12	96,2 %	34,28 %

La diferencia es enorme: un hombre de 70 años aparece con 91 % de riesgo en el mundo artificial 50/50 pero con ≈ 17 % en la población. Del mismo modo, con la sensibilidad y especificidad del modelo, el **valor predictivo positivo esperado a prevalencia 2 % sería solo 6,7 %**: la mayoría de las alertas serían falsas. Un modelo útil para priorizar, no para diagnosticar; ignorar este ajuste es un error frecuente al llevar modelos entrenados con datos balanceados a producción.

6.12.4 Ejercicio 3 · K vecinos más cercanos (K-NN)

Enunciado

Estudie el algoritmo de K-Nearest con todo detalle, mejore su documentación y con base en él haga cambios para una aplicación.

Documentación del algoritmo

K-NN (Cover y Hart, 1967) es el ejemplo típico de **aprendizaje basado en instancias** o «perezoso»: no construye un modelo explícito; guarda el conjunto de entrenamiento y difiere todo el trabajo al momento de predecir. Para un punto nuevo $\mathbf{x}$: (1) calcula su distancia a todos los ejemplos guardados; (2) selecciona los k más cercanos, $N_k(\mathbf{x})$; (3) decide por votación, opcionalmente ponderada:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in N_k(\mathbf{x})} w_i \mathbf{1}[y_i = c], \quad w_i = 1 \text{ (uniforme)} \quad \text{o} \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i) + \epsilon} \text{ (ponderado)}$$

La fracción ponderada de vecinos positivos es además una estimación de $P(y = 1 | \mathbf{x})$. Las piezas de diseño son:

- **La distancia.** Minkowski de orden p : $d_p(\mathbf{x}, \mathbf{z}) = (\sum_a |x_a - z_a|^p)^{1/p}$; $p = 2$ es la euclídea, $p = 1$ la Manhattan. La distancia *define* qué significa «parecido»: es el verdadero modelo.

- **La escala.** Una variable con rango 0-100 (edad) domina a una binaria 0-1. Por eso se suele estandarizar con media y desviación calculadas *solo* en entrenamiento.
- **k controla el sesgo y la varianza.** $k = 1$ memoriza (acierto de entrenamiento $\approx 100\%$, frontera muy irregular); k grande suaviza hasta predecir siempre la clase mayoritaria. Con dos clases se prefiere k impar para evitar empates.
- **La maldición de la dimensionalidad.** En muchas dimensiones todas las distancias se parecen y los «vecinos» dejan de ser cercanos; variables irrelevantes agregan ruido a la distancia (motivación de la reducción de dimensionalidad, sección 6.6.2).
- **Costo.** Entrenar es $O(1)$; predecir cuesta $O(n \cdot d)$ por consulta (aquí vectorizado con NumPy). Para n grande se usan árboles KD o búsqueda aproximada de vecinos.

Verificación de la implementación

En el data set Wine (178 vinos, 13 variables, 3 clases) se compararon 16 configuraciones ($k \in \{1, 3, 5, 11\}$, $p \in \{1, 2\}$, voto uniforme y ponderado) contra `sklearn.neighbors.KNeighborsClassifier`: las predicciones coincidieron en el **100 %** de los casos en todas ellas (exactitud entre 96,3 % y 100,0 %). En el problema de COVID la variante estandarizada también coincidió al 100 % con `sklearn`.

Cambios introducidos para la aplicación

Los datos de COVID mezclan variables numéricas (edad, demora) y categóricas (sexo, tipo de contagio, asintomático, departamento). Se implementaron y compararon cuatro variantes, cada una con su k elegido por validación cruzada:

- **Estandarización interna** con parámetros del entrenamiento (cambio 1).
- **Voto ponderado por el inverso de la distancia** (cambio 2): los vecinos más cercanos pesan más y se rompen empates.
- **Distancia HEOM** (*Heterogeneous Euclidean-Overlap Metric*, Wilson y Martínez, 1997) (cambio 3): para cada atributo numérico usa $|x_a - z_a| / \text{rango}_a$ y para cada categórico 0 si coinciden y 1 si no. Así el departamento cuenta como *una* variable (con one-hot contaría doble, porque dos departamentos distintos difieren en dos columnas).
- **Umbral de decisión ajustable** sobre la probabilidad estimada (cambio 4), equivalente al costo asimétrico de la SVM.

$$d_{HEOM}(\mathbf{x}, \mathbf{z}) = \sqrt{\sum_a d_a(x_a, z_a)^2}, \quad d_a = \frac{|x_a - z_a|}{\text{rango}_a} \text{ (numérico)}, \quad d_a = \mathbf{1}[x_a \neq z_a] \text{ (categórico)}$$

Tabla 6.12.11. Variantes de K-NN en el problema de COVID-19.

Variante	k elegido	Exactitud CV (media $\pm$ desv.)	Exactitud prueba	Sensibilidad	Especificidad
Euclídea sin estandarizar	11	83,8 % $\pm$ 1,9 %	83,0 %	85,0 %	81,0 %
Euclídea estandarizada	9	83,2 % $\pm$ 2,4 %	81,5 %	86,0 %	77,0 %
Estandarizada + voto ponderado	17	83,3 % $\pm$ 1,8 %	81,0 %	80,0 %	82,0 %
HEOM + voto ponderado	7	83,8 % $\pm$ 2,6 %	80,5 %	80,0 %	81,0 %

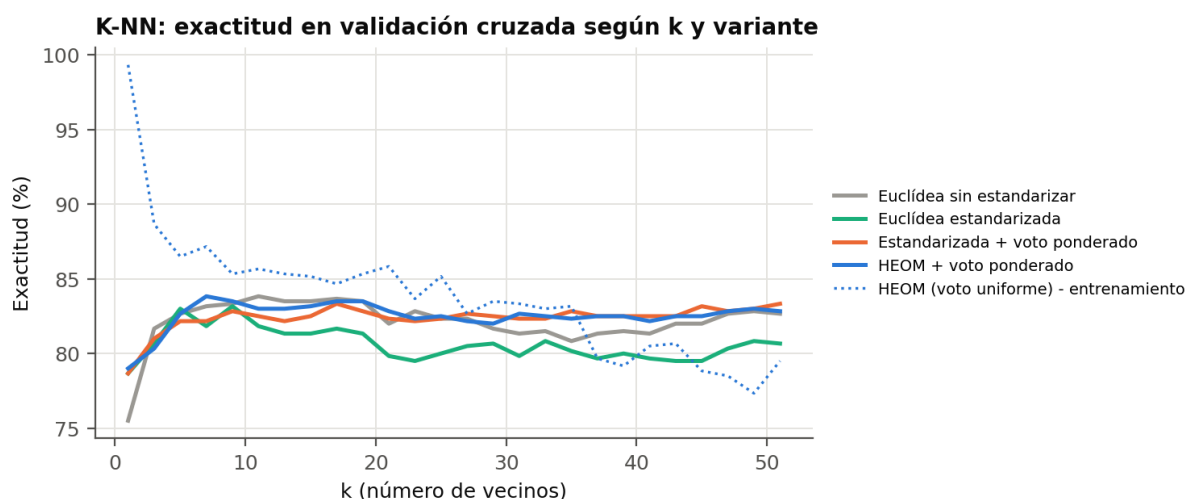


Figura 6.12.6. Exactitud en validación cruzada según k para cada variante. La línea punteada es la exactitud sobre el propio entrenamiento: con $k = 1$ es casi 100 % (memorización) y cae al aumentar k.

Interpretación: ¿por qué gana la versión «sin estandarizar»?

El resultado contradice la receta habitual y es instructivo. Sin estandarizar, la edad (rango 0-100) domina por completo la distancia euclídea: el modelo se vuelve, en la práctica, «K-NN sobre la edad». Como la edad es la variable con casi toda la señal, esa ponderación accidental es buena. Al estandarizar, las variables binarias débiles (departamento, sexo) pesan lo mismo que la edad y agregan ruido a la distancia. Las diferencias entre variantes (≈ 1 punto) están dentro de la desviación entre pliegues (≈ 2 puntos), así que ninguna es estadísticamente superior. La lección es que en K-NN **elegir y ponderar variables es tan importante como elegir k**.

Los vecinos explican la predicción. K-NN permite mostrar por qué decidió. Para el primer caso de prueba (mujer de 42 años, contagio relacionado, demora 15 días, Antioquia; recuperada), sus 7 vecinos tienen entre 29 y 58 años, todos son recuperados y *todos* tienen demora de exactamente 15 días en Antioquia. El modelo acierta, pero en parte gracias al artefacto de imputación detectado en 6.12.2: la explicación por vecinos ayuda a descubrir esos atajos.

Tabla 6.12.12. Efecto del umbral de decisión (K-NN HEOM ponderado, $k = 7$).

Umbral sobre P(fallece)	Exactitud	Sensibilidad	Especificidad	Precisión
0,3	80,5 %	87,0 %	74,0 %	77,0 %
0,4	78,5 %	82,0 %	75,0 %	76,6 %
0,5	80,5 %	80,0 %	81,0 %	80,8 %
0,6	81,0 %	77,0 %	85,0 %	83,7 %

Bajar el umbral a 0,3 sube la sensibilidad de 80 % a 87 %. Tiempo de predicción de los 200 casos: 9,8 ms.

6.12.5 Ejercicio 4 · Árboles de decisión

Enunciado

Estudie el algoritmo de árboles de decisión, con todo detalle mejore su documentación y con base en él haga cambios para una aplicación.

Documentación del algoritmo

Un árbol de decisión representa el modelo como una secuencia de preguntas simples organizadas jerárquicamente: cada nodo interno pregunta «¿ $x_a \leq u$?», cada rama es una respuesta y cada hoja asigna una clase (y una probabilidad). Se construye de arriba hacia abajo con una estrategia voraz: en cada nodo se prueba cada variable y cada umbral posible (puntos medios entre valores consecutivos ordenados) y se elige el corte que más reduce la **impureza**.

$$\text{Gini}(t) = 1 - \sum_c p_c^2 \quad H(t) = -\sum_c p_c \log_2 p_c \quad \Delta I = I(t) - \frac{n_L}{n} I(t_L) - \frac{n_R}{n} I(t_R)$$

Con la entropía H , ΔI es la **ganancia de información** de ID3/C4.5 (Quinlan); con Gini es el criterio de CART (Breiman et al., 1984). Ambas valen 0 en un nodo puro y son máximas con clases mezcladas por igual (Gini 0,5; $H = 1$ bit en dos clases). En la práctica producen árboles muy parecidos; Gini es más barato porque no calcula logaritmos.

Cuándo parar. Si se deja crecer, el árbol llega a hojas puras y memoriza el entrenamiento. Hay dos defensas: *pre-poda* (profundidad máxima, mínimo de muestras por hoja, ganancia mínima) y *post-poda*. La implementada es la **poda por costo-complejidad** de CART, que penaliza el número de hojas $|T|$:

$$R_\alpha(T) = R(T) + \alpha |T|, \quad g(t) = \frac{R(t) - R(T_t)}{|T_t| - 1}$$

Para cada nodo interno t , $g(t)$ mide cuánto riesgo evita su subárbol T_t por cada hoja adicional que usa. Se colapsa repetidamente el nodo de menor g (el «eslabón más débil»), lo que produce una secuencia de árboles anidados, uno por cada α ; el α final se elige por validación cruzada.

Ventajas: interpretable (se lee como reglas), no requiere escalar variables, maneja interacciones y es rápido (construcción $O(d \cdot n \log n)$ por nivel). **Desventajas:** alta varianza (pequeños cambios en los datos cambian el árbol), fronteras escalonadas paralelas a los ejes y tendencia al sobreajuste; de ahí nacen los bosques aleatorios y el boosting.

Verificación de la implementación

Tabla 6.12.13. Árbol propio frente a sklearn.tree.DecisionTreeClassifier (Breast Cancer, prueba de 171 casos).

Criterio	Prof. máx.	Hojas (propio / sklearn)	Exactitud propio	Exactitud sklearn	Acuerdo
Gini	2	4 / 4	90,6 %	90,6 %	100,0 %
Gini	4	12 / 12	91,2 %	90,6 %	99,4 %
Gini	sin límite	16 / 16	91,8 %	90,6 %	98,8 %
Entropía	2	4 / 4	88,3 %	88,3 %	100,0 %
Entropía	4	9 / 9	93,0 %	93,0 %	100,0 %
Entropía	sin límite	12 / 12	92,4 %	93,0 %	98,2 %

El número de hojas coincide en todos los casos. Las pocas diferencias de predicción se deben a empates: cuando dos variables ofrecen exactamente la misma ganancia, sklearn las recorre en orden aleatorio y la implementación propia en orden fijo. La ruta de poda es idéntica: los 12 valores de α coinciden con los de `cost_complexity_pruning_path` con diferencia máxima de $1,2 \cdot 10^{-16}$.

Aplicación: riesgo de fallecimiento por COVID-19

El árbol sin restricciones sobreajusta. Creció hasta 14 niveles y 137 hojas, acierta el 99,3 % en entrenamiento, pero solo el 76,8 % en validación cruzada y el 79,0 % en prueba: peor que la regla manual de una línea. La figura 6.12.7 muestra la brecha al crecer la profundidad.

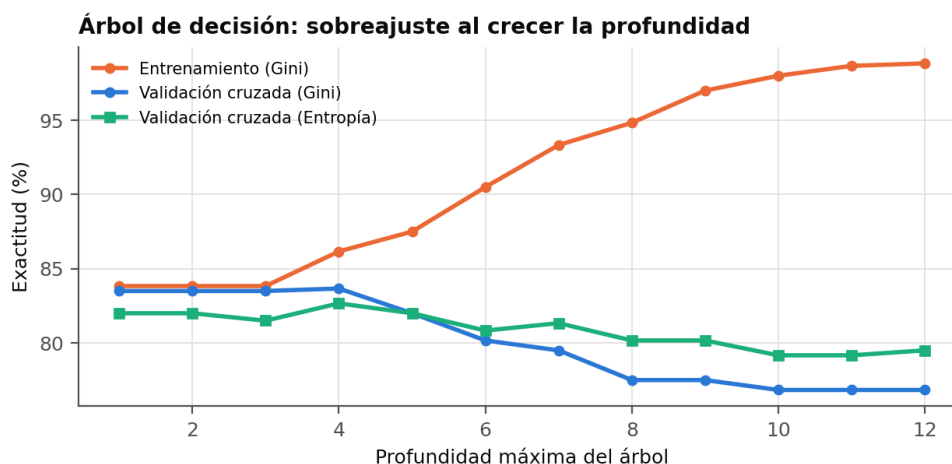


Figura 6.12.7. Exactitud de entrenamiento y de validación cruzada según la profundidad máxima.

Poda por costo-complejidad. Se evaluaron por validación cruzada todos los α de la ruta de poda. El máximo se alcanzó con $\alpha = 0,0074$, que deja un árbol de **4 hojas y profundidad 2** (exactitud CV 83,7 % $\pm$ 2,0 %). La regla de «una desviación estándar» (el árbol más simple con CV dentro de un error estándar del mejor) llevaría a un tocón de 2 hojas, prácticamente la regla manual.

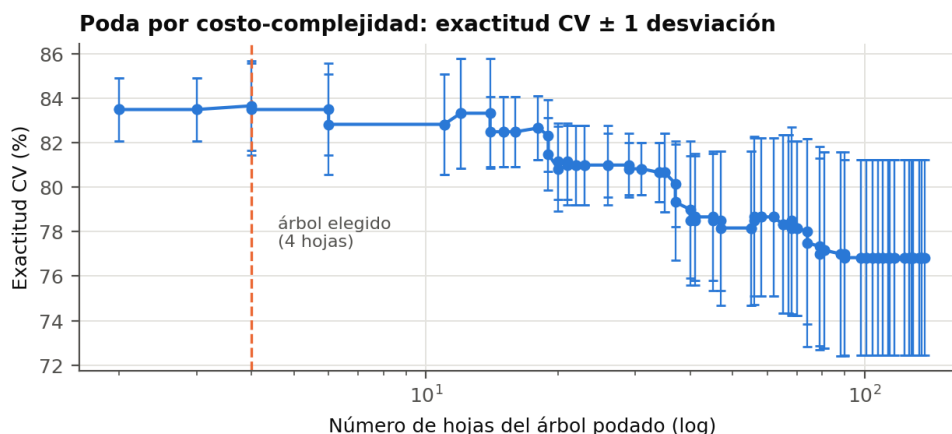


Figura 6.12.8. Exactitud de validación cruzada ($\pm$ 1 desviación entre pliegues) para los árboles de la ruta de poda. Menos hojas generalizan mejor en este problema.

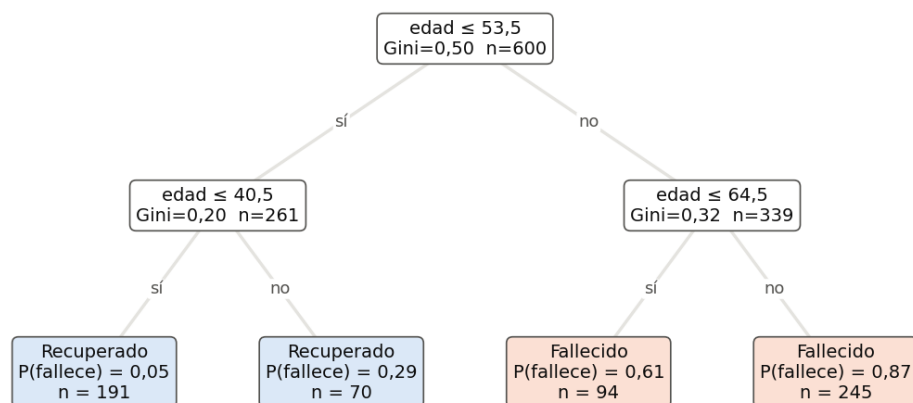


Figura 6.12.9. Árbol final podado (4 hojas). Cada hoja muestra la probabilidad estimada de fallecer y el número de casos de entrenamiento.

Cambios introducidos para la aplicación

Cambio 1 · Reglas SI-ENTONCES en español

Se añadió un exportador que recorre el árbol y escribe una regla por hoja. Esto responde a la preocupación de la sección 6.11 (y de Cathy O'Neil) sobre modelos opacos e incontestables: un árbol podado puede auditarse línea por línea.

Tabla 6.12.14. Reglas extraídas del árbol podado.

#	SI ...	ENTONCES	P(clase)	Casos
1	edad ≤ 53,5 Y edad ≤ 40,5	Recuperado	95,3 %	191
2	edad ≤ 53,5 Y edad > 40,5	Recuperado	71,4 %	70
3	edad > 53,5 Y edad ≤ 64,5	Fallecido	60,6 %	94
4	edad > 53,5 Y edad > 64,5	Fallecido	87,3 %	245

Las reglas son clínicamente sensatas: el riesgo crece por escalones con la edad (5 %, 29 %, 61 % y 87 % en la muestra balanceada). La importancia de variables del árbol podado asigna el 100 % de la reducción de impureza a la edad.

Cambio 2 · Pesos por clase

Tabla 6.12.15. Árbol podado con pesos por clase (conteos ponderados en la impureza).

Peso de «fallecido»	Exactitud	Sensibilidad	Especificidad	FN	FP
1,0	81,0 %	90,0 %	72,0 %	10	28
1,5	81,0 %	90,0 %	72,0 %	10	28
2,0	81,0 %	90,0 %	72,0 %	10	28
3,0	81,0 %	92,0 %	70,0 %	8	30

En un árbol tan pequeño los pesos solo mueven los umbrales cuando son grandes (peso 3): el efecto es más escalonado que en la SVM porque las predicciones son constantes por hoja.

Cambio 3 · Poda con α elegido por validación cruzada

Pasar de 137 a 4 hojas subió la exactitud de prueba de 79,0 % a 81,0 % y la sensibilidad de 78,0 % a 90,0 %.

Lo que revela un árbol más profundo: correlación no es causalidad

Un árbol de profundidad 3 (mínimo 15 casos por hoja) introduce una segunda variable: para personas de 41 a 53 años, una demora de diagnóstico *mayor* a 12,5 días se asocia con *menor* riesgo (84 % recuperados frente a 61 %). Clínicamente no tiene sentido: es el rastro de los 15 días imputados a recuperados en la fuente. El árbol encontró una correlación real en los datos pero sin relación causal, justo el problema que el capítulo discute con *The Book of Why* (sección 6.11). La validación cruzada descartó esas ramas y la interpretabilidad permitió entender por qué.

6.12.6 Comparación de los algoritmos y conclusiones

Tabla 6.12.16. Resultados en el conjunto de prueba (200 casos). VPP a 2 %: valor predictivo positivo esperado si la prevalencia fuera del 2 %.

Modelo	Exactitud	Sensibilidad	Especificidad	Precisión	F1	VPP a 2 %
Regla manual (edad ≥ 54)	81,0 %	90,0 %	72,0 %	76,3 %	0,826	6,2 %
SVM RBF	82,5 %	91,0 %	74,0 %	77,8 %	0,839	6,7 %
K-NN HEOM (k=7)	80,5 %	80,0 %	81,0 %	80,8 %	0,804	7,9 %
Árbol podado	81,0 %	90,0 %	72,0 %	76,3 %	0,826	6,2 %

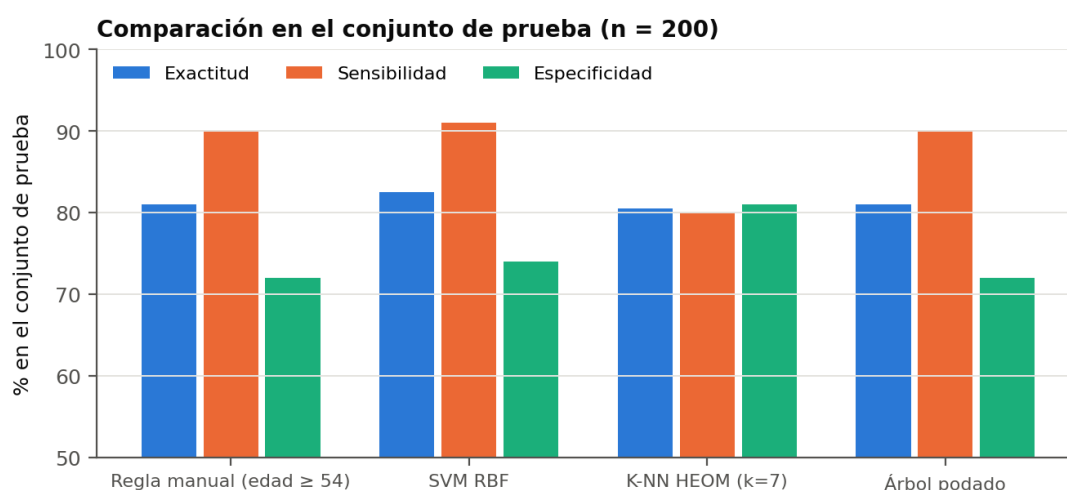


Figura 6.12.10. Exactitud, sensibilidad y especificidad en el conjunto de prueba.

Con 200 casos de prueba, una diferencia de un punto porcentual son dos pacientes; el intervalo de confianza del 95 % de una exactitud de 82 % es de aproximadamente ± 5 puntos. Ningún modelo es significativamente mejor que otro ni que la regla manual. Esta es la conclusión más importante del ejercicio, y conecta con dos ideas del capítulo:

- **La irrazonable efectividad de los datos (6.7, Banko y Brill).** Algoritmos muy distintos convergen al mismo desempeño cuando comparten la misma información. Aquí el techo lo pone el data set: sin comorbilidades, signos vitales ni laboratorios, la edad explica casi todo lo que puede explicarse. Mejorar requeriría *mejores variables*, no un algoritmo más sofisticado.
- **La solución actual como línea base (6.10).** Una regla de una línea («edad ≥ 54 ») es transparente, no necesita software y rinde igual. Un proyecto de ML debe demostrar valor frente a ella; aquí el valor está en la probabilidad calibrada, en el control explícito del costo de los errores y en la auditoría de sesgos, no en la exactitud.

Comparación cualitativa

Tabla 6.12.17. Comparación cualitativa de los tres algoritmos, a partir de lo observado en los ejercicios.

Aspecto	SVM	K-NN	Árbol de decisión
Qué aprende	Frontera de margen máximo en el espacio del núcleo	Nada: guarda los ejemplos	Particiones jerárquicas por umbrales
Hiperparámetros clave	C, γ , núcleo	k, distancia, pesos	Profundidad, mínimo por hoja, α de poda
Escalado de variables	Imprescindible	Imprescindible (y decisivo)	Innecesario
Datos mixtos	Vía codificación	Distancia HEOM	Naturalmente
Interpretabilidad	Baja (RBF); pesos si es lineal	Media: muestra los vecinos	Alta: reglas legibles
Costo de entrenar / predecir	Alto / bajo (solo vectores de soporte)	Nulo / alto (todos los ejemplos)	Bajo / muy bajo
Salida probabilística	Requiere Platt	Fracción de vecinos	Proporción en la hoja
Riesgo principal	Mala elección de C y γ	Variables irrelevantes, dimensionalidad	Sobreajuste sin poda

Conclusiones generales

- **Ejercicio 1:** ML puede redescubrir una regla a partir de ejemplos si su espacio de hipótesis la contiene (regresión polinomial exacta), pero cuando la regla se conoce, calcularla directamente es miles de veces más barato y siempre exacto. La red neuronal aproxima bien dentro del rango de entrenamiento y falla fuera de él.
- **Implementaciones propias:** las tres coinciden con scikit-learn (SVM: mismos vectores de soporte y sesgo; K-NN: 100 % de acuerdo; árbol: misma ruta de poda), lo que valida la documentación matemática del Anexo A.
- **Cambios para la aplicación:** el costo asimétrico, los umbrales ajustables, la calibración y la corrección por prevalencia son los que convierten un clasificador académico en una herramienta útil para triaje. Sin la corrección por prevalencia, las razones de probabilidad (odds) quedarían infladas unas 49 veces.
- **Datos antes que algoritmos:** el análisis exploratorio encontró un sesgo temporal de muestreo y un artefacto de imputación (15 días) que los modelos aprovecharon. Detectarlos exigió mirar los datos y los modelos interpretables, en línea con las secciones 6.7.6 y 6.11 del capítulo.
- **Ética:** un modelo de riesgo basado casi solo en la edad podría usarse para discriminar en la asignación de recursos. Estos resultados son un ejercicio académico con una muestra de 800 casos y no deben usarse para decisiones clínicas.

Referencias

- Breiman, L., Friedman, J., Olshen, R. y Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.
- Banko, M. y Brill, E. (2001). Scaling to very very large corpora for natural language disambiguation. *Proc. ACL*.
- Cortes, C. y Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273-297.
- Cover, T. y Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Trans. Information Theory*, 13(1), 21-27.
- Fan, R.-E., Chen, P.-H. y Lin, C.-J. (2005). Working set selection using second order information for training SVM. *JMLR*, 6, 1889-1918.
- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly.
- Instituto Nacional de Salud. Casos positivos de COVID-19 en Colombia. Portal de Datos Abiertos, www.datos.gov.co (recurso gt2j-8ykr).
- Keerthi, S., Shevade, S., Bhattacharyya, C. y Murthy, K. (2001). Improvements to Platt's SMO algorithm for SVM classifier design. *Neural Computation*, 13(3), 637-649.
- Martínez P., J. J. (2026). *6. Introducción a Machine Learning*. Material del curso.
- Pearl, J. y Mackenzie, D. (2018). *The Book of Why*. Basic Books.

- Platt, J. (1998). Sequential minimal optimization: a fast algorithm for training support vector machines. Microsoft Research, MSR-TR-98-14.
- Platt, J. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*.
- Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81-106.
- Wilson, D. R. y Martínez, T. R. (1997). Improved heterogeneous distance functions. *Journal of Artificial Intelligence Research*, 6, 1-34.

Anexo A · Código fuente documentado

Implementaciones propias usadas en los ejercicios 2 a 4, con su documentación matemática. Solo dependen de NumPy. Los scripts de experimentación (generación del data set del ejercicio 1, validación cruzada y figuras) se entregan junto con este documento.

A.1 svm_desde_cero.py · SVM con SMO, costo por clase, Platt y corrección por prevalencia

```
"""
Máquina de Vectores de Soporte (SVM) implementada desde cero.

Problema primal (margen suave, con costo por clase):
    min_{w,b,x_i} 1/2 ||w||^2 + sum_i C_i x_i
    s.a.          y_i (w·phi(x_i) + b) >= 1 - x_i,    x_i >= 0,    y_i en {-1, +1}

Problema dual (el que se resuelve):
    min_a 1/2 a^T Q a - e^T a,    Q_ij = y_i y_j K(x_i, x_j)
    s.a.    0 <= a_i <= C_i,    y^T a = 0

Solucionador: SMO (Sequential Minimal Optimization) con selección del "par de máxima
violación" de las condiciones KKT (Keerthi et al. 2001; Fan, Chen y Lin 2005, base de
LIBSVM). En cada iteración se optimizan analíticamente solo dos multiplicadores.

Función de decisión: f(x) = sum_i a_i y_i K(x_i, x) + b ; clase = signo(f(x)).

CAMBIOS introducidos para la aplicación (triaje de riesgo de COVID-19):
1. Costo asimétrico por clase (C_i = C * peso[y_i]): penaliza más los falsos negativos.
2. Escalado de Platt: convierte f(x) en una probabilidad P(y=1|x) = 1/(1+exp(A f + B)).
3. Corrección por prevalencia: la muestra está balanceada (50 %), pero en la población
   la letalidad es mucho menor; se reajustan las probabilidades con el teorema de Bayes.
"""
import numpy as np

# ----- núcleos
def kernel_lineal(X1, X2, **_):
    return X1 @ X2.T

def kernel_rbf(X1, X2, gamma=0.1, **_):
    """K(x, z) = exp(-gamma ||x - z||^2)"""
    d2 = (X1**2).sum(1)[:, None] + (X2**2).sum(1)[None, :] - 2 * X1 @ X2.T
    return np.exp(-gamma * np.maximum(d2, 0.0))

KERNELS = {"lineal": kernel_lineal, "rbf": kernel_rbf}

class SVM:
    """SVM binaria. Etiquetas de entrada en {0,1} o {-1,+1}."""

    Parámetros
    -----
    C : float          costo de violar el margen (C grande -> margen duro, riesgo de sobreajuste)
    kernel : str        'lineal' o 'rbf'
    gamma : float       ancho del núcleo RBF (gamma grande -> fronteras más irregulares)
    peso_clase : dict    {etiqueta: peso}; multiplica C para esa clase (cambio 1)
    tol : float         tolerancia KKT para detener SMO
    max_iter : int      máximo de iteraciones SMO
    """

    def __init__(self, C=1.0, kernel="rbf", gamma=0.1, peso_clase=None, tol=1e-3, max_iter=100_000):
        self.C, self.kernel, self.gamma = C, kernel, gamma
        self.peso_clase, self.tol, self.max_iter = peso_clase, tol, max_iter

    # ----- entrenamiento
    def _K(self, A, B):
        return KERNELS[self.kernel](A, B, gamma=self.gamma)

    def fit(self, X, y):
        X = np.asarray(X, float)
        y = np.asarray(y)
        self.clases_ = np.unique(y)
        ys = np.where(y == self.clases_[1], 1.0, -1.0) # a {-1,+1}
        n = len(ys)
        Cv = np.full(n, float(self.C))
        if self.peso_clase:
            for etiqueta, w in self.peso_clase.items():
                Cv[y == etiqueta] *= w
        K = self._K(X, X)
        Q = (ys[:, None] * ys[None, :]) * K
        a = np.zeros(n)
        G = -np.ones(n) # gradiente de la función dual: Q a - e
        it = 0
        while it < self.max_iter:
            # --- selección del par de máxima violación (WSS1)
            menos_yG = -ys * G
            arriba = ((ys > 0) & (a < Cv)) | ((ys < 0) & (a > 0)) # I_up
            abajo = ((ys < 0) & (a < Cv)) | ((ys > 0) & (a > 0)) # I_low
            if not arriba.any() or not abajo.any():
                break
            i = np.where(arriba)[0][np.argmax(menos_yG[arriba])]
            j = np.where(abajo)[0][np.argmin(menos_yG[abajo])]
            if menos_yG[i] - menos_yG[j] < self.tol: # KKT satisfechas
```

```

        break
# --- actualización analítica de (a_i, a_j)
ai_old, aj_old = a[i], a[j]
Ci, Cj = Cv[i], Cv[j]
quad = K[i, i] + K[j, j] - 2 * K[i, j]
quad = quad if quad > 1e-12 else 1e-12
if ys[i] != ys[j]:
    delta = (-G[i] - G[j]) / quad
    diff = a[i] - a[j]
    a[i] += delta; a[j] += delta
    if diff > 0:
        if a[j] < 0: a[j] = 0; a[i] = diff
    else:
        if a[i] < 0: a[i] = 0; a[j] = -diff
    if diff > Ci - Cj:
        if a[i] > Ci: a[i] = Ci; a[j] = Ci - diff
    else:
        if a[j] > Cj: a[j] = Cj; a[i] = Cj + diff
else:
    delta = (G[i] - G[j]) / quad
    s = a[i] + a[j]
    a[i] -= delta; a[j] += delta
    if s > Ci:
        if a[i] > Ci: a[i] = Ci; a[j] = s - Ci
    else:
        if a[j] < 0: a[j] = 0; a[i] = s
    if s > Cj:
        if a[j] > Cj: a[j] = Cj; a[i] = s - Cj
    else:
        if a[i] < 0: a[i] = 0; a[j] = s
    G += Q[:, i] * (a[i] - ai_old) + Q[:, j] * (a[j] - aj_old)
    it += 1
self.n_iter_ = it
# --- sesgo b a partir de los vectores de soporte libres (0 < a < C)
yG = ys * G
libres = (a > 1e-8) & (a < Cv - 1e-8)
if libres.any():
    rho = yG[libres].mean()
else:
    menos_yG = -ys * G
    arriba = ((ys > 0) & (a < Cv)) | ((ys < 0) & (a > 0))
    abajo = ((ys < 0) & (a < Cv)) | ((ys > 0) & (a > 0))
    rho = -(menos_yG[arriba].max() + menos_yG[abajo].min()) / 2
self.b_ = -rho
sv = a > 1e-8
self.vectores_soporte_ = X[sv]
self.coef_dual_ = (a * ys)[sv] # a_i y_i
self.n_sv_ = int(sv.sum())
self.alpha_ = a
return self

# ----- predicción
def decision_function(self, X):
    return self._K(np.asarray(X, float), self.vectores_soporte_) @ self.coef_dual_ + self.b_

def predict(self, X):
    return np.where(self.decision_function(X) >= 0, self.clases_[1], self.clases_[0])

# ----- cambio 2: Platt
def calibrar(self, X, y, iter_newton=100, f=None):
    """Ajusta  $P(y=1|f) = 1/(1+\exp(A \cdot f + B))$  por máxima verosimilitud (Newton-Raphson),
    con los objetivos suavizados de Platt (1999) para evitar sobreajuste.
    Si se pasa `f` (valores de decisión fuera de pliegue, obtenidos por validación
    cruzada) se usan esos en lugar de recalcularlos sobre los mismos datos de ajuste."""
    f = self.decision_function(X) if f is None else np.asarray(f, float)
    y = (np.asarray(y) == self.clases_[1]).astype(float)
    n1, n0 = y.sum(), len(y) - y.sum()
    t = np.where(y == 1, (n1 + 1) / (n1 + 2), 1 / (n0 + 2))
    A, B = 0.0, np.log((n0 + 1) / (n1 + 1))
    for _ in range(iter_newton):
        p = 1 / (1 + np.exp(A * f + B))
        g = np.array([(t - p) * f].sum(), (t - p).sum()) # gradiente de -logL
        w = p * (1 - p)
        H = np.array([(w * f * f).sum(), (w * f).sum()], [(w * f).sum(), w.sum()]) + 1e-10 * np.eye(2)
        paso = np.linalg.solve(H, g)
        A, B = A - paso[0], B - paso[1]
        if np.abs(paso).max() < 1e-10:
            break
    self.platt_ = (A, B)
    return self

def predict_proba(self, X, prevalencia=None, prevalencia_entrenamiento=0.5):
    """ $P(y=1|x)$ . Con `prevalencia` aplica el cambio 3 (corrección de probabilidades a priori):
     $\text{odds\_real} = \text{odds\_modelo} * [\pi/(1-\pi)] / [\pi_{\text{ent}}/(1-\pi_{\text{ent}})]$ """
    A, B = self.platt_
    p = 1 / (1 + np.exp(A * self.decision_function(X) + B))
    if prevalencia is not None:

```

```
pi, pi0 = prevalencia, prevalencia_entrenamiento
odds = p / (1 - p) * (pi / (1 - pi)) / (pi0 / (1 - pi0))
p = odds / (1 + odds)
return p
```

A.2 knn_desde_cero.py · K-NN con Minkowski, HEOM y voto ponderado

```
"""
```

K vecinos más cercanos (K-NN) implementado desde cero.

Idea: no hay fase de entrenamiento propiamente dicha (aprendizaje "perezoso"): se guardan los ejemplos y, para un punto nuevo x, se buscan los k ejemplos más cercanos según una distancia $d(x, x_i)$ y se decide por votación.

```
y_hat(x) = argmax_c sum_{i en N_k(x)} w_i * 1[y_i = c]
```

```
- voto uniforme:      w_i = 1
- voto ponderado:     w_i = 1 / (d(x, x_i) + eps)      (cambio 2)
```

Distancias disponibles:

```
- Minkowski de orden p:  d(x,z) = ( sum_a |x_a - z_a|^p )^(1/p)  (p=1 Manhattan, p=2 Euclídea)
- HEOM (Heterogeneous Euclidean-Overlap Metric, Wilson y Martínez 1997)  (cambio 3)
  d(x,z) = sqrt( sum_a d_a(x_a, z_a)^2 )
  d_a = |x_a - z_a| / rango_a      si el atributo a es numérico
  d_a = 0 si x_a = z_a, 1 si no    si el atributo a es categórico
```

CAMBIO5 introducidos para la aplicación (datos mixtos de COVID-19):

1. Estandarización interna (z-score) de atributos numéricos con parámetros del entrenamiento: sin ella la edad (0-100) dominaría a las variables binarias (0-1).
2. Voto ponderado por el inverso de la distancia (vecinos cercanos pesan más y se rompen los empates).
3. Distancia HEOM para mezclar atributos numéricos y categóricos sin que la codificación one-hot duplique el peso de una variable categórica.
4. Probabilidad estimada = fracción (ponderada) de vecinos positivos -> permite mover el umbral de decisión.

```
"""
```

```
import numpy as np
```

```
class KNN:
```

```
def __init__(self, k=5, p=2, pesos="uniforme", metrica="minkowski",
             estandarizar=True, categoricas=None, eps=1e-9):
    self.k, self.p, self.pesos, self.metrica = k, p, pesos, metrica
    self.estandarizar, self.categoricas, self.eps = estandarizar, categoricas or [], eps
```

```
def fit(self, X, y):
    X = np.asarray(X, float)
    self.y_ = np.asarray(y)
    self.clases_ = np.unique(self.y_)
    n_atr = X.shape[1]
    self.es_cat_ = np.zeros(n_atr, bool)
    self.es_cat_[self.categoricas] = True
    if self.metrica == "heom":
        rango = X.max(0) - X.min(0)
        self.escala_ = np.where(self.es_cat_, 1.0, np.where(rango > 0, rango, 1.0))
        self.centro_ = np.zeros(n_atr)
    elif self.estandarizar:
        self.centro_ = X.mean(0)
        s = X.std(0)
        self.escala_ = np.where(s > 0, s, 1.0)
    else:
        self.centro_, self.escala_ = np.zeros(n_atr), np.ones(n_atr)
    self.X_ = (X - self.centro_) / self.escala_
    return self

def _distancias(self, Xq):
    Xq = (np.asarray(Xq, float) - self.centro_) / self.escala_
    dif = np.abs(Xq[:, None, :] - self.X_[None, :, :]) # (n_consulta, n_ent, n_atr)
    if self.metrica == "heom":
        dif[:, :, self.es_cat_] = (dif[:, :, self.es_cat_] > 0).astype(float)
        return np.sqrt((dif**2).sum(-1))
    if self.p == np.inf:
        return dif.max(-1)
    return (dif**self.p).sum(-1) ** (1.0 / self.p)
```

```
def vecinos(self, Xq):
    D = self._distancias(Xq)
    idx = np.argsort(D, axis=1, kind="stable")[:, :self.k]
    return idx, np.take_along_axis(D, idx, 1)
```

```
def predict_proba(self, Xq):
    idx, d = self.vecinos(Xq)
    w = np.ones_like(d) if self.pesos == "uniforme" else 1.0 / (d + self.eps)
    votos = np.stack([(w * (self.y_[idx] == c)).sum(1) for c in self.clases_], 1)
    return votos / votos.sum(1, keepdims=True)
```

```
def predict(self, Xq, umbral=None):
    P = self.predict_proba(Xq)
    if umbral is not None and len(self.clases_) == 2:
        return np.where(P[:, 1] >= umbral, self.clases_[1], self.clases_[0])
    return self.clases_[np.argmax(P, 1)]
```

A.3 arbol_desde_cero.py · CART con Gini/entropía, poda costo-complejidad y reglas

```
"""
Árbol de decisión CART (Classification And Regression Trees) implementado desde cero.

Construcción recursiva ("divide y vencerás"): en cada nodo se busca la pregunta binaria
    ¿x_a <= umbral?
que más reduce la impureza de los hijos, ponderada por su tamaño:

    Ganancia(a, u) = I(padre) - [ n_izq/n * I(izq) + n_der/n * I(der) ]

Medidas de impureza para una distribución de clases p_c:
    Gini:      I = 1 - sum_c p_c^2
    Entropía:  I = - sum_c p_c log2 p_c      (la ganancia es la "ganancia de información" de ID3/C4.5)

Criterios de parada (pre-poda): profundidad máxima, mínimo de muestras para dividir,
mínimo de muestras por hoja, ganancia mínima.

Post-poda por costo-complejidad (Breiman et al. 1984):
    R_alpha(T) = R(T) + alpha * |hojas(T)|
Para cada nodo interno t se calcula el "eslabón más débil"
    g(t) = [R(t) - R(T_t)] / (|hojas(T_t)| - 1)
y se colapsan sucesivamente los nodos de menor g(t). Se obtiene una secuencia de árboles
anidados, uno por cada alpha; el alpha se elige por validación cruzada.
(R se mide con la impureza ponderada, igual que scikit-learn, para poder comparar.)

CAMBIOS introducidos para la aplicación:
    1. Pesos por clase (costo asimétrico): cada ejemplo aporta peso[y] a los conteos.
    2. Poda por costo-complejidad con alpha elegido por validación cruzada.
    3. Exportación de reglas SI-ENTONCES legibles en español (el modelo deja de ser una
       "caja negra", ver sección 6.11 del capítulo) y cálculo de importancia de variables.
"""
import copy
import numpy as np

def _impureza(conteos, criterio):
    tot = conteos.sum()
    if tot <= 0:
        return 0.0
    p = conteos / tot
    if criterio == "gini":
        return 1.0 - (p**2).sum()
    p = p[p > 0]
    return float(-(p * np.log2(p)).sum())

class Nodo:
    __slots__ = ("atributo", "umbral", "izq", "der", "valor", "n", "peso", "impureza", "prof")

    def __init__(self, valor, n, peso, impureza, prof):
        self.atributo = None; self.umbral = None; self.izq = None; self.der = None
        self.valor, self.n, self.peso, self.impureza, self.prof = valor, n, peso, impureza, prof

    @property
    def es_hoja(self):
        return self.izq is None

class ArbolDecision:
    def __init__(self, criterio="gini", max_prof=None, min_muestras_division=2,
                 min_muestras_hoja=1, ganancia_min=0.0, peso_clase=None, ccp_alpha=0.0):
        self.criterio, self.max_prof = criterio, max_prof
        self.min_div, self.min_hoja, self.ganancia_min = min_muestras_division, min_muestras_hoja, ganancia_min
        self.peso_clase, self.ccp_alpha = peso_clase, ccp_alpha

    # ----- construcción
    def fit(self, X, y, nombres=None):
        X = np.asarray(X, float); y = np.asarray(y)
        self.clases_ = np.unique(y)
        self.nombres_ = list(nombres) if nombres is not None else [f"x{j}" for j in range(X.shape[1])]
        yi = np.searchsorted(self.clases_, y)
        w = np.ones(len(y))
        if self.peso_clase:
            for c, pc in self.peso_clase.items():
                w[y == c] = pc
        self.n_atr_ = X.shape[1]
        self.peso_total_ = w.sum()
        self.importancia_bruta_ = np.zeros(self.n_atr_)
        self.raiz_ = self._crecer(X, yi, w, 0)
        if self.ccp_alpha > 0:
            self.raiz_ = self._podar_hasta(self.raiz_, self.ccp_alpha)
        self._calc_importancia()
        return self

    def _conteos(self, yi, w):
```

```

        return np.bincount(yi, weights=w, minlength=len(self.clases_))

def _crecer(self, X, yi, w, prof):
    cont = self._conteos(yi, w)
    nodo = Nodo(cont, len(yi), w.sum(), _impureza(cont, self.criterio), prof)
    if (self.max_prof is not None and prof >= self.max_prof) or len(yi) < self.min_div \
        or nodo.impureza == 0.0:
        return nodo
    mejor = self._mejor_division(X, yi, w, nodo)
    if mejor is None:
        return nodo
    a, u = mejor
    m = X[:, a] <= u
    nodo.atributo, nodo.umbral = a, u
    nodo.izq = self._crecer(X[m], yi[m], w[m], prof + 1)
    nodo.der = self._crecer(X[~m], yi[~m], w[~m], prof + 1)
    return nodo

def _mejor_division(self, X, yi, w, nodo):
    n, K = len(yi), len(self.clases_)
    Y1 = np.zeros((n, K)); Y1[np.arange(n), yi] = w # conteos ponderados one-hot
    mejor_gan, mejor = self.ganancia_min, None
    tot = nodo.valor
    for a in range(X.shape[1]):
        orden = np.argsort(X[:, a], kind="mergesort")
        xs = X[orden, a]
        izq = np.cumsum(Y1[orden], 0)[-1] # conteos a la izquierda de cada corte
        der = tot - izq
        n_izq = np.arange(1, n)
        validos = (xs[1:] > xs[:-1]) & (n_izq >= self.min_hoja) & (n - n_izq >= self.min_hoja)
        if not validos.any():
            continue
        wi, wd = izq.sum(1), der.sum(1)
        if self.criterio == "gini":
            Ii = 1 - ((izq / np.maximum(wi, 1e-300))[:, None])**2).sum(1)
            Id = 1 - ((der / np.maximum(wd, 1e-300))[:, None])**2).sum(1)
        else:
            pi = izq / np.maximum(wi, 1e-300)[:, None]; pd_ = der / np.maximum(wd, 1e-300)[:, None]
            Ii = -(np.where(pi > 0, pi * np.log2(np.where(pi > 0, pi, 1)), 0)).sum(1)
            Id = -(np.where(pd_ > 0, pd_ * np.log2(np.where(pd_ > 0, pd_, 1)), 0)).sum(1)
        gan = nodo.impureza - (wi * Ii + wd * Id) / nodo.peso
        gan[~validos] = -np.inf
        k = int(np.argmax(gan))
        if gan[k] > mejor_gan + 1e-12:
            mejor_gan = gan[k]
            mejor = (a, (xs[k] + xs[k + 1]) / 2.0)
    return mejor

# ----- poda costo-complejidad
def _R(self, nodo): # riesgo del nodo como hoja
    return nodo.impureza * nodo.peso / self.peso_total_

def _subarbol(self, nodo): # (R(T_t), número de hojas)
    if nodo.es_hoja:
        return self._R(nodo), 1
    ri, hi = self._subarbol(nodo.izq); rd, hd = self._subarbol(nodo.der)
    return ri + rd, hi + hd

def _eslabon_mas_debil(self, raiz):
    mejor = [np.inf, []]
    def rec(t):
        if t.es_hoja:
            return
        RT, h = self._subarbol(t)
        g = (self._R(t) - RT) / (h - 1)
        if g < mejor[0] - 1e-15:
            mejor[0], mejor[1] = g, [t]
        elif abs(g - mejor[0]) <= 1e-15:
            mejor[1].append(t)
        rec(t.izq); rec(t.der)
    rec(raiz)
    return mejor

def _podar_hasta(self, raiz, alpha):
    raiz = copy.deepcopy(raiz)
    while not raiz.es_hoja:
        g, nodos = self._eslabon_mas_debil(raiz)
        if g > alpha:
            break
        for t in nodos:
            t.izq = t.der = None; t.atributo = t.umbral = None
    return raiz

def ruta_poda(self):
    """Secuencia de alphas efectivos y número de hojas de cada árbol anidado."""
    raiz = copy.deepcopy(self.raiz_)
    alphas, hojas = [0.0], [self._subarbol(raiz)[1]]

```

```

while not raiz.es_hoja:
    g, nodos = self._eslabon_mas_debil(raiz)
    for t in nodos:
        t.izq = t.der = None
        alphas.append(max(g, 0.0)); hojas.append(self._subarbol(raiz)[1])
return np.array(alphas), np.array(hojas)

# ----- predicción
def _hoja(self, x):
    t = self.raiz_
    while not t.es_hoja:
        t = t.izq if x[t.atributo] <= t.umbral else t.der
    return t

def predict_proba(self, X):
    X = np.asarray(X, float)
    V = np.array([self._hoja(x).valor for x in X])
    return V / V.sum(1, keepdims=True)

def predict(self, X):
    return self.clases_[np.argmax(self.predict_proba(X), 1)]

# ----- interpretación
def _calc_importancia(self):
    imp = np.zeros(self.n_atr_)
    def rec(t):
        if t.es_hoja:
            return
        imp[t.atributo] += t.peso * t.impureza - t.izq.peso * t.izq.impureza - t.der.peso * t.der.impureza
        rec(t.izq); rec(t.der)
    rec(self.raiz_)
    self.importancia_ = imp / imp.sum() if imp.sum() > 0 else imp

def profundidad(self):
    def rec(t):
        return 0 if t.es_hoja else 1 + max(rec(t.izq), rec(t.der))
    return rec(self.raiz_)

def n_hojas(self):
    return self._subarbol(self.raiz_)[1]

def reglas(self, etiquetas=None, fmt=None):
    """Lista de reglas SI-ENTONCES, una por hoja."""
    etiquetas = etiquetas or {c: str(c) for c in self.clases_}
    fmt = fmt or {}
    salida = []
    def cond(nombre, op, u):
        f = fmt.get(nombre)
        if f == "bin":
            return f"{nombre} = {'0' if op == '<=' else '1'}"
        return f"{nombre} {op} {u:.1f}".replace(".", ",")
    def rec(t, conds):
        if t.es_hoja:
            p = t.valor / t.valor.sum()
            k = int(np.argmax(p))
            salida.append(dict(condiciones=conds, clase=etiquetas[self.clases_[k]],
                               prob=float(p[k]), n=int(t.n)))
            return
        nom = self.nombres_[t.atributo]
        rec(t.izq, conds + [cond(nom, "<=", t.umbral)])
        rec(t.der, conds + [cond(nom, ">", t.umbral)])
    rec(self.raiz_, [])
    return salida

```

A.4 datos.py · Preparación del data set del INS y métricas

```
"""
Carga y preparación del data set gubernamental usado en los ejercicios 2, 3 y 4.

Fuente: Instituto Nacional de Salud (INS) - "Casos positivos de COVID-19 en Colombia",
portal de Datos Abiertos del Estado colombiano, www.datos.gov.co, recurso gt2j-8ykr
(Ley 1712 de 2014). Muestra balanceada de 800 casos: 100 fallecidos y 100 recuperados
de cada uno de 4 departamentos (Bogotá, Antioquia, Valle, Atlántico).

Tarea: clasificación binaria y = 1 (fallecido) vs y = 0 (recuperado).
"""

import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

RUTA = "/home/claude/data/covid_ins_muestra.csv"
SEMILLA = 42
DEPTOS = ["ANTIOQUIA", "ATLANTICO", "BOGOTA", "VALLE"]

def cargar_crudo():
    df = pd.read_csv(RUTA)
    return df

def preparar(df, mediana_demora=None):
    """Limpieza + ingeniería de características (secciones 6.7.3 y 6.7.4 del capítulo).

    - edad: se convierte a años según unidad_medida (1 años, 2 meses, 3 días).
    - sexo_M: fuente_relacionado: variables binarias (0/1).
    - asintomatico: 1 si no hay fecha de inicio de síntomas (dato faltante informativo).
    - demora_dx: días entre inicio de síntomas y diagnóstico; los faltantes se asignan
    con la MEDIANA del conjunto de entrenamiento (asignación por mediana, 6.7.3.2.2).
    - departamento: codificación one-hot (4 columnas).
    """
    X = pd.DataFrame()
    X["edad"] = df["edad"].astype(float)
    X["sexo_M"] = (df["sexo"].str.upper() == "M").astype(int)
    X["fuente_relacionado"] = (df["fuente"] == "R").astype(int)
    X["asintomatico"] = df["asintomatico"].astype(int)
    if mediana_demora is None:
        mediana_demora = float(df["demora_dx"].median())
    X["demora_dx"] = df["demora_dx"].fillna(mediana_demora).clip(0, 60)
    for d in DEPTOS:
        X[f"dep_{d.title()}"] = (df["departamento"] == d).astype(int)
    y = df["fallecido"].astype(int).values
    return X, y, mediana_demora

def particion():
    """Partición estratificada 75 % entrenamiento / 25 % prueba.
    La validación se hace con validación cruzada de 5 pliegues DENTRO del entrenamiento."""
    df = cargar_crudo()
    df_tr, df_te = train_test_split(df, test_size=0.25, stratify=df["fallecido"], random_state=SEMILLA)
    Xtr, ytr, med = preparar(df_tr)
    Xte, yte, _ = preparar(df_te, med)
    return Xtr, Xte, ytr, yte

COLUMNAS_NUM = ["edad", "demora_dx"]

def metricas_binarias(y, yp):
    """Matriz de confusión (figura 6.2) y medidas derivadas."""
    y = np.asarray(y); yp = np.asarray(yp)
    VP = int(((y == 1) & (yp == 1)).sum()); VN = int(((y == 0) & (yp == 0)).sum())
    FP = int(((y == 0) & (yp == 1)).sum()); FN = int(((y == 1) & (yp == 0)).sum())
    n = VP + VN + FP + FN
    exact = (VP + VN) / n
    sens = VP / (VP + FN) if VP + FN else 0.0
    esp = VN / (VN + FP) if VN + FP else 0.0
    prec = VP / (VP + FP) if VP + FP else 0.0
    f1 = 2 * prec * sens / (prec + sens) if prec + sens else 0.0
    return dict(VP=VP, VN=VN, FP=FP, FN=FN, exactitud=exact, sensibilidad=sens,
                especificidad=esp, precision=prec, F1=f1)

def vpp_prevalencia(sens, esp, prev):
    """Valor predictivo positivo esperado si la prevalencia real es `prev` (teorema de Bayes)."""
    return sens * prev / (sens * prev + (1 - esp) * (1 - prev))

if __name__ == "__main__":
    Xtr, Xte, ytr, yte = particion()
    print(Xtr.shape, Xte.shape, ytr.mean(), yte.mean())
    print(Xtr.describe().T)
```